数据结构 A 第 7 次作业

姓名: 王子川　　**学号:** PB25000055

1(5.8) 下标对应

由图中的存储顺序可知,

$$k = 2(i-1) + ((j+1) \bmod 2).$$

2(5.26)

思路是按照行指针逐行遍历, 将十字链表中的非零元素及其下标按行优先顺序存入三元组顺序表中.

```
#define MAXSIZE 100

typedef int ElemType;
typedef int status;

#define OK 1
#define ERROR 0

typedef struct OLNode {
    int i, j;
    ElemType e;
    struct OLNode *right;
    struct OLNode *down;
} OLNode, *OLink;

typedef struct CrossList {
    OLink *rhead;
    OLink *chead;
    int mu, nu, tu;
} CrossList;

typedef struct triple {
    int i, j;
    ElemType e;
} triple;

typedef struct TSMatrix {
    triple data[MAXSIZE];
    int mu, nu, tu;
} TSMatrix;

status CrossListToTSMatrix(CrossList A, TSMatrix &B) {
    int tu = 0;

    for (int i = 0; i < A.mu; i++) {
        OLink p = A.rhead[i];

        while (p != nullptr) {
            if (tu >= MAXSIZE)
                return ERROR;

            B.data[tu].i = p->i;
```

```
            B.data[tu].j = p->j;
            B.data[tu].e = p->e;

            tu++;
            p = p->right;
        }
    }

    B.mu = A.mu;
    B.nu = A.nu;
    B.tu = tu;

    return OK;
}
```

时间复杂度为 $O(t_u)$, 其中 t_u 为非零元素的个数.

3(5.30) 表头表尾递归求解广义表深度

采用递归的算法思想, 将广义表分解为表头和表尾, 分别求其深度. 基本情况是表为空表时, 深度为 1; 若表头为原子, 则其深度为 0. 对于非空广义表 L, 有

$$Depth(L) = \max\{Depth(Head(L)) + 1, Depth(Tail(L))\}.$$

```
int GListDepth(GList L) {
    if (!L)
        return 1;

    GList head = GetHead(L);
    GList tail = GetTail(L);

    int headDepth;

    if (!head)
        headDepth = 1;
    else if (head->tag == ATOM)
        headDepth = 0;
    else
        headDepth = GListDepth(head);

    int tailDepth = GListDepth(tail);

    return max(headDepth + 1, tailDepth);
}
```

对于有限且无递归的广义表, 每次递归均作用于原表的真子结构, 因此算法能够终止. 时间复杂度与广义表的结构规模成线性关系, 即每个相关结点至多访问一次.

4(5.32) 递归判断两个表是否相等

延续上一题的思路, 递归比较两个广义表的表头和表尾. 若两个结点均为空, 则二者相等; 若一个为空而另一个非空, 则二者不相等. 对于非空结点, 首先比较其类型; 若均为原子, 则比较原子值; 若均为表, 则递归比较其表头和表尾.

```
bool GListSame(GList A, GList B) {
    if (!A || !B)
```

```
3         return A == B;
4
5     if (A->tag != B->tag)
6         return false;
7
8     if (A->tag == ATOM)
9         return A->atom == B->atom;
10
11    return GListSame(A->ptr.hp, B->ptr.hp)
12        && GListSame(A->ptr.tp, B->ptr.tp);
13 }
```

在有限、无共享、无递归的情况下, 两个广义表中的每个相关结点至多访问一次, 因此时间复杂度为

$$O(N_A + N_B),$$

其中 N_A, N_B 分别为广义表 A, B 的结构规模.